

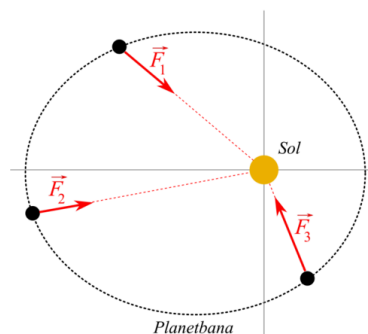
<https://towardsdatascience.com/use-python-to-create-two-body-orbits-a68aed78099c/>
https://en.wikipedia.org/wiki/Classical_central-force_problem
https://en.wikipedia.org/wiki/Central_force

Introduction

- This is a problem in classical mechanics that predicts the motion of two point masses under mutual gravitational attraction
- Since we are ignoring everything that can affect these two bodies of mass besides each other, the only thing that can affect them is each other's gravitational pull
- For the restricted version of this problem, in orbital mechanics, one of the masses is considered to have negligible mass, causing it to have no effect on the other mass. This is how we will simulate the Restricted Two Body problem in Python. However, in reality, the smaller mass does have an effect on the larger mass.
- In other words, we must factor gravity when making our Python program. The larger body of mass will be stationary while the smaller body of mass will lack gravitational pull.
- Models closer to reality include other forces affecting a spacecraft such as other natural bodies (other planets, moons, etc.), solar radiation, oblateness and aerodynamic drag (if its orbit is touching an atmosphere).

Physics behind the RESTRICTED problem

- To visualize the model, we begin with the stationary body. The smaller body will be influenced by this larger body. We will be, then, analyzing the motion of the smaller body with respect to the larger mass.
- This motion will be defined by the distance and the influence of a central force. A central force must drive particles either directly towards or directly away from a fixed point in space, the center of force, labeled O.
- Since we are simulating gravity, the central force will drive the particles toward the fixed point in space.



Wikipedia

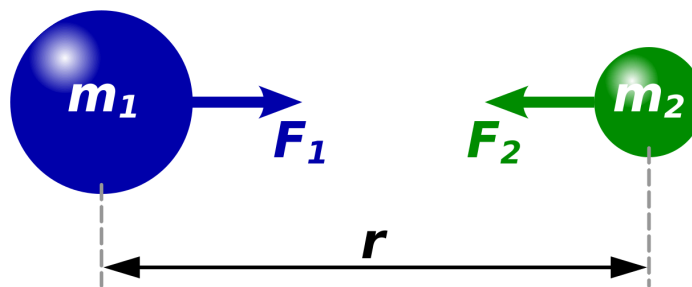
- In classical mechanics, a central force on an object is a force that is directed towards or away from a point called center of force.

$$\mathbf{F}(\mathbf{r}) = F(\mathbf{r})\hat{\mathbf{r}}$$

- where $\mathbf{F}$ is a force vector, F is a scalar valued force function (whose absolute value gives the magnitude of the force and is positive if the force is outward and negative if the force is inward), $\mathbf{r}$ is the position vector, $\|\mathbf{r}\|$ is its length, and $\hat{\mathbf{r}} = \mathbf{r}/\|\mathbf{r}\|$ is the corresponding unit vector.

Deriving equations of motion for simulation

- For our program to work, we will need to specify our central force. Since we are simulating orbital mechanics, our central force will be gravity. We will define this force using Newton's Law of Universal Gravitation, which states that every particle of matter in the universe attracts every other particle with a force directly proportional to the product of their masses and inversely proportional to the square of the distance between their centers:



$$F_1 = F_2 = G \frac{m_1 \times m_2}{r^2}$$

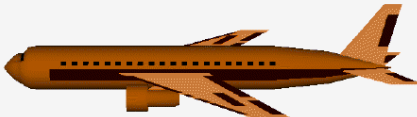
- In our case:
 - G is the gravitational constant.
 - m_1 is the mass of our large, stationary body (like the Sun or a planet).
 - m_2 is the mass of our smaller body (like a smaller planet, spacecraft or satellite).
 - r is the scalar distance between both masses.
- For our simulation, we will need to build an Inertial Reference Frame, which is a coordinate system in which Newton's laws of motion hold true without the need to account for fictitious forces. In this state, an object with no net external force acting on it remains at rest or moves with a constant velocity (zero acceleration).
- We will place the origin (0,0) of our coordinate system exactly at the center of the heavy body of mass (m_1). This means that the bigger mass will stay completely stationary while we track the position and velocity of the smaller mass (m_2) relative to it.
- Moreover, to find out how this force changes the motion of the smaller mass, we apply Newton's Second Law of Motion, which states that the acceleration of an object is directly proportional to the net force acting on it and inversely proportional to its mass:



Newton's Second Law

Definitions

Glenn
Research
Center



Differential Form:

With mass constant:

Force = change of momentum
with change of time

Force = mass X acceleration

Force = mass X change in velocity with time

$$F = \frac{d(mv)}{dt}$$

$$F = m a$$

$$F = \frac{m (V_1 - V_0)}{(t_1 - t_0)}$$

*Force, acceleration, momentum and velocity are all vector quantities.
Each has both a magnitude and a direction.*

- Since gravity will be the only net force acting on our smaller mass, we can substitute our gravitational force equation into Newton's Second Law.

$$m_2 \cdot a = -G \frac{m_1 \times m_2}{r^3} r$$

- The negative sign at the beginning of the equation is critical in astrodynamics. In our coordinate system, the position vector r points outward from the center of the bigger mass to the smaller mass. Because gravity is an exclusively attractive force, it pulls the smaller mass inward toward the planet. The negative sign is the mathematical way of stating that the acceleration vector always points in the exact opposite direction of the position vector, pulling the object back toward the origin rather than pushing it into deep space.
- Because the mass of the spacecraft (m_2) appears on both sides of the equation, it cancels out completely:

$$\ddot{r} = - \frac{G \cdot m_1}{r^3} r$$

- This proves a fundamental principle: the acceleration of the object does not depend on its own mass. To clean up our calculations in Python, we can combine G and m_1 into a single constant known as the Standard Gravitational Parameter, labeled as μ , which is the product of the universal gravitational constant and our larger mass:

$$\mu = G \cdot m_1$$

- Substituting μ gives us the final vector equation of motion that you often see in astrodynamics literature:

$$\ddot{\mathbf{r}} = -\frac{\mu}{r^3} \mathbf{r}$$

- (We make the assumption that the mass of the smaller body is negligible compared to the major body ($m_2 \ll m_1$)).

2D components for Python

- The equations of motion for a negligible mass moving under the influence of gravity of a larger mass are as follows:

$$\begin{aligned}\ddot{x} &= -\frac{\mu}{(x^2 + y^2 + z^2)^{3/2}} x \\ \ddot{y} &= -\frac{\mu}{(x^2 + y^2 + z^2)^{3/2}} y \\ \ddot{z} &= -\frac{\mu}{(x^2 + y^2 + z^2)^{3/2}} z\end{aligned}$$

- Now that we have our equations of motion, we need to convert them to a state form that can be numerically integrated. The state form includes the position and velocity vector at a certain time. Here is the state and its time derivative.

$$\begin{pmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ \dot{z} \end{pmatrix} \quad \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \ddot{x} = -\frac{\mu}{r^3} x \\ \ddot{y} = -\frac{\mu}{r^3} y \\ \ddot{z} = -\frac{\mu}{r^3} z \end{pmatrix}$$

- The state time derivative will be made into a function that can be numerically integrated. Python has built-in numerical integrators that we can utilize for this. To create our orbit, we will be using `odeint` function from the SciPy package.